

Week 6 Reflection
TCSS-498 Directed Reading (Raghavi Sakpal)
Seth Campbell

This week has been trying to bring Forward Kinematics online. This process is still ongoing. This is where the clean theory of the Denavit-Hartenberg approach (rubber) meets the reality of even slightly complex design considerations (road). I've previously set up my rotation matrix and aligned my identity matrix for each joint relative to each other, thinking all that's left are the DH values. But some questions arose while trying to align the first joint to the second. There's a mechanical offset to give it a simpler rotation. How do I account for this?

In most other cases of people attempting this project, they use of Euler angles, a vector representation to relate the joints. This is all well and good, but now how does this work with my current understanding of forward kinematics? After some digging in the text, it works relatively well. The Euler angle expands the projection matrix to a 4x4, and this gets multiplied by the rotation matrix instead – also expanded to a 4x4 matrix. The resulting matrix is the new transformation matrix. The only other major adjustment is the projection matrix; this matrix now has to relate to the Euler representation and not the axis of rotation of the joint. This alone was a lot to wrap my head around, and I kept thinking of what I wrote in the week 1 reflection about 'not wanting to get too deep into the math'. It sounds so off-base now.

I took time to rethink my process for the Forward Kinematics and my layout. I initially had thought of having the Frame sit in between ServoCommunication and ESPNow classes with Leader handling communication to and from, and Transform (now Forward_K) would sit off to the side only interacting with frame to calculate end effect position. But I ran into an issue with timing where the Servo Communication would write to the frame at the same time as the ESPNow Leader would direct and package contents from the same array for ESPNow transmitting. This caused not infrequent issues. I've since reverted back to my previous layout and just keep Frame and Forward_K off to the side. In doing this, however, I've introduced a bug I'm trying to work through. The signal (and thus teleoperation) pipeline is now fractured again. I have clean signals being pulled and transmitted from the leader to the follower, the follower correctly processes them, but when trying to read and write to the servos, I can do one or the other, but not both. I do get clean signal processing when I plug into the board directly, but not when using the ESP32, I've ordered another set of ESP32 C3, and it won't take more than a few minutes to swap once they arrive on Tuesday. I hope it's a simple error on a \$7 board.

In the meantime, since my leader is still functioning properly, I need to address a problem with the position being presented by my FK process. Currently, I'm getting roughly accurate magnitudes on horribly skewed axes. Currently, my z-axis spans mostly where the x-axis should be but is tilted, and the x and y-axes don't seem to be orthogonal to the z-axis but are to each other. Hopefully, I can correct the FK issue before the spring colloquium and, at a minimum, have teleoperation online with coordinates displayed.